
Can Agent Benchmarks Support Their Scores?

Evidence-Supported Bounds for Interactive-Agent Evaluation

Anonymous Author(s)

Affiliation

Address

email

Abstract

Interactive-agent benchmarks map an agent run to a binary outcome via outcome checks. When checks rely on surface-level signals or miss the agent’s action path, they fail to determine whether the run succeeded. For example, a benchmark case asks whether Alice’s shipping address changed, but its outcome check only verifies that the agent clicked Save. This does not guarantee that the intended state change was applied; the agent may have updated the wrong record. Counting such a run as a success makes the score misleading. Benchmark quality therefore depends on outcome detection as well as task design.

We address this by adding an outcome-evidence reporting layer to existing benchmarks, without changing their tasks, agents, or evaluators. The layer does three things: (i) specifies, before scoring, which stored artifacts would verify the claimed outcome for each case; (ii) applies these requirements to each completed record and assigns one of three evidence states: *supported success*, *supported failure*, or *unresolved*; and (iii) reports evidence-supported bounds that quantify uncertainty from unresolved records. Records that remain *unresolved* are kept visible rather than silently counted, dropped, or hidden inside a single success rate.

We evaluate this outcome-evidence layer on four public benchmarks: AGENTDOJO, τ^3 -bench retail, APPWORLD, and WEBARENA-VERIFIED. The resulting reports make outcome-evidence quality visible alongside task quality, supporting more evidence-grounded agent evaluation and uncertainty reporting. We release the case-level requirements, reason labels, audit checks, robust-ranking summaries, and rescorer needed to reproduce the reports.

1 Introduction

Interactive-agent benchmarks increasingly ask agents to navigate interfaces, call tools, and change persistent environment state. Their headline result is usually a single success rate. But a reported success is only as reliable as the evidence used to verify it. If a task asks an agent to update Alice’s shipping address, a trace showing that the agent clicked Save is not the same as evidence that Alice’s backend record changed. The same action trace is compatible with the correct record being updated, the wrong record being modified, or no durable state change occurring at all.

This is an outcome-evidence gap. The benchmark claim is a binary statement about what happened in the environment, while the stored artifacts may include only screenshots, action logs, final messages, partial verifier inputs, or other indirect traces. The gap is especially important for interactive agents because correctness often depends on identity resolution, side effects, post-state queries, policy constraints, and multi-step tool interactions. When these artifacts are insufficient, a point score can overstate what the benchmark has actually verified.

We address this gap with an outcome-evidence reporting layer. The layer is additive: it leaves benchmark tasks, agents, environments, and native evaluators unchanged. For each case, it writes a short list of evidence needed before scoring, specifying which stored artifacts would decide the benchmark’s own success claim. Each completed record is then reported as *supported success*, *supported failure*, or *unresolved*. An *unresolved* record is not treated as a success, not converted into a failure, and not dropped from the evaluation set.

When a record is unresolved, none of the usual shortcuts is neutral. Counting it as a success overstates what the evidence supports. Counting it as a failure can punish missing instrumentation rather than agent behavior. Dropping it changes which records are evaluated and can create agent-dependent post-selection, because some agents leave more decisive traces than others. We therefore keep unresolved records in the same set of completed records and report the range of all-record performance still compatible with the stored evidence. The counted-only score is still useful, but only as a diagnostic conditional on decidability, not as a headline all-record estimate.

We evaluate this reporting layer on four public interactive-agent benchmarks. AGENTDOJO and τ^3 -bench retail provide evidence-sensitive settings where plausible traces can outrun stored evidence. APPWORLD and WEBARENA-VERIFIED use strong state-based or verified evaluators and serve as calibration controls. The study is designed to be falsifiable: large unresolved shares on the controls would indicate over-conservative evidence requirements, while narrow control bounds and wider evidence-sensitive bounds would show that the layer distinguishes weak stored evidence from strong native verification.

The paper makes four contributions: (i) we formalize the outcome-evidence gap in interactive-agent benchmarks, where a binary success claim may be reported even when stored artifacts are insufficient to decide that claim; (ii) we introduce case-level evidence requirements tied to each benchmark’s own success claim, plus a three-state counting rule that separates *supported success*, *supported failure*, and *unresolved* records without changing benchmark tasks, agents, environments, or native evaluators; (iii) we derive evidence-supported performance bounds over the fixed set of completed records, making the counted-only score a diagnostic over decidable records rather than a headline all-record estimate; and (iv) we apply the method to four public benchmarks, showing that evidence uncertainty is concentrated in evidence-sensitive domains while remaining narrow on strong state-based evaluator controls, and release the case-level requirements, case cards, reason labels, audit checks, robust-ranking summaries, and rescorer needed to reproduce the reports.

2 Related Work and Preliminaries

Interactive-agent evaluation and outcome checks. Interactive-agent benchmarks evaluate agents that navigate interfaces, call tools, and alter environment state. Relevant environments include WebArena [17], BrowserGym [1], ToolBench-style tool-use benchmarks [9], AGENTDOJO [2], τ^3 -bench retail [12], ANDROIDWORLD [10], WORKARENA [3], and OSWORLD-VERIFIED [14]. These benchmarks make interactive evaluation realistic, but realism alone does not guarantee that a stored trace decides the outcome claim being reported.

Verified evaluators and benchmark validity. A recent line of work strengthens outcome checks through state-based or verified evaluation, including the verified WEBARENA-VERIFIED release [5], APPWORLD database-state unit tests [13], and SWE-bench Verified [8]. Benchmark-validity work, including the Agentic Benchmark Checklist, argues that construct validity, outcome validity, and reporting assumptions should be made explicit [18]. Our contribution is complementary: rather than proposing a new environment, a stronger domain evaluator, or an audit checklist, we add a cross-benchmark reporting layer that asks what the stored artifacts of completed runs can support under each benchmark’s own claim. LLM-as-judge methods [16, 11] and benchmark refresh or contamination studies [6, 4, 15] are useful diagnostics, but they do not replace evidence that a native outcome claim is decidable from stored artifacts.

Claims, records, traces, and evidence. A *case* specifies a task, the initial environment, the benchmark claim being tested, and the evidence required to decide that claim. A benchmark claim is the binary outcome statement the benchmark reports for a completed run, such as whether a target record was updated, a message was sent, or a constraint was violated. The *native evaluator* is the benchmark’s released mechanism for turning its available artifacts into a success or failure label.

We use *case unit* only for a small bookkeeping issue: in most domains, one task instance is one case unit, while in paired-arm settings such as AGENTDOJO, two runnable arms form one case unit even though they are executed separately. Decisions live at the level of *records*. A record is one agent’s result on one case unit. The distinction matters because the same case unit can be countable for one agent, whose trace exposes decisive evidence, and UNRESOLVE for another, whose trace does not.

An *episode* is one concrete execution of one agent on one runnable arm of a case. Episodes produce *traces*: ordered records of actions, observations, tool calls, browser events, messages, files, and post-run artifacts. We use *stored artifacts* for the full set of traces, logs, state snapshots, receipts, verifier inputs, and post-run measurements retained after execution. *Evidence* is the subset of those artifacts that can support the active benchmark claim. A trace can be plausible without being decisive.

Staying tied to the benchmark claim. Because the method sits on top of existing benchmarks, the requirements must not silently make the task harder. For each case unit, we write down: (i) the benchmark’s own binary claim; (ii) the official source of that claim, such as the task text, evaluator code, policy, or schema; (iii) the artifacts needed to decide success or fail; (iv) the rule for assigning UNRESOLVE; and (v) whether any requirement goes beyond the benchmark’s own claim.

The requirements must be minimal with respect to the benchmark’s own semantics. For example, if a task says only “update the address” and the native evaluator only checks the target record’s address, then the requirements ask for target-record identity and post-state evidence. A no-collateral-change requirement is included only if the benchmark’s own task, policy, evaluator, or reported claim requires it. Otherwise it is labeled as a stronger measurement claim and reported separately.

When sources differ, the requirements follow a fixed source hierarchy: official evaluator semantics first; official task text and policy second; and schema constraints needed to interpret evaluator-visible state third. We do not add requirements from annotator intuition alone. Any requirement not grounded in one of these sources is labeled as a stronger measurement claim and excluded from the main bounds tied to the benchmark’s own claim.

3 Method

The method is a post-run reporting pass over completed benchmark runs. For each record, it asks one narrow question: do the stored artifacts decide the benchmark’s own outcome claim? The inputs are the benchmark’s official sources and the artifacts already saved for each run. The output is a measurement report with counts, coverage, bounds, ranking decisions, and reason labels. The pass does not change the task, agent, environment, native evaluator, or native score.

Post-run procedure. The procedure is simple. We first run the benchmark normally, with the same tasks, prompts, tools, environment, and released evaluator. Before scoring stored runs, we write one short requirement for each case unit: the benchmark claim in plain language, the artifacts that would decide success or failure, and the rule for leaving the record undecided. These requirements are derived from the official source hierarchy in Section 2. This is the main guardrail: a requirement may not add a stronger task claim unless it is labeled and reported separately.

A scoring pass then applies the fixed requirements to the stored run artifacts. Each completed record receives one of three labels. S is used when the artifacts support success under the benchmark claim; F when they support failure; U when they do not decide the claim. Agent-caused invalid actions, timeouts, tool misuse, or aborts after task start are failures when the released benchmark would fail the run; they are not moved into U .

Counting and bounds. The report then uses only these counts. For agent a in domain d , let $N_{a,d}$ be the number of completed records in the evaluation set, and let $S_{a,d}$, $F_{a,d}$, and $U_{a,d}$ be the counts of the three labels, so $N_{a,d} = S_{a,d} + F_{a,d} + U_{a,d}$. The intuition is simple: there are three piles. The counted-only score asks, among records the artifacts decide, what fraction are successes. The bound asks a different question: over all completed records, what performance values are still consistent with the stored evidence? For readability, write S, F, U, N for the four counts in this cell. We report

$$\text{CountedScore}_r(a, d) = \frac{S}{S + F}, \quad \text{Perf}_r(a, d) \in \left[\frac{S}{N}, \frac{S + U}{N} \right], \quad \text{Width}_r(a, d) = \frac{U}{N}.$$

The lower endpoint counts only supported successes; the upper endpoint treats every unresolved record as a possible success. The width is therefore not an extra statistic but the unresolved share itself. When $U = 0$, the bound collapses to the all-record score; when $U > 0$, any single point estimate must hide a decision about records the stored artifacts did not decide. These are not confidence intervals; they are partial-identification bounds that report what the observed evidence can and cannot identify [7]. Other scores and labels are diagnostic views over the same traces; the headline object is the evidence-supported bound.

Diagnostics. The first diagnostic separates evidence uncertainty from finite-case variation. A bound computed on a fixed sample of cases may change if the benchmark sampled a different set of cases from the same domain. To show that sensitivity, we use case-cluster bootstrap intervals: each replicate resamples case units within a domain and keeps all agent records attached to the resampled case unit. We do not use record-level binomial standard errors. These intervals condition on the stored traces and fixed agent versions; they do not measure uncertainty from repeated executions, API drift, tool nondeterminism, or environment volatility.

The second diagnostic handles leaderboards. A native point score may put agent i above agent j , but that ordering is not supported if unresolved records could reverse it. We therefore compare agents by the bounds themselves. For agents i and j in domain d , the difference interval implied by the bounds is $\Delta(i, j; d) = [\text{LB}_{i,d} - \text{UB}_{j,d}, \text{UB}_{i,d} - \text{LB}_{j,d}]$. The decision rule is directional: $i \succ j$ if $\text{LB}_{i,d} > \text{UB}_{j,d}$, and $j \succ i$ if $\text{LB}_{j,d} > \text{UB}_{i,d}$. If neither inequality holds, the pair is *not identified*: the stored evidence does not rule out either agent being at least as good as the other. Ordinary interval overlap is reported as “?” rather than as a tie. For an observed ordering $i \succ j$, the dominance margin is $M_{i \succ j} = \text{LB}_{i,d} - \text{UB}_{j,d}$. We add an asterisk only when the 2.5th percentile of the case-cluster bootstrap distribution of $M_{i \succ j}$ remains positive. The symmetric criterion applies when $j \succ i$. The asterisk is a descriptive stability tag, not a formal significance test.

The third diagnostic explains unresolved records. The width U/N says how much of the evaluation is not decided by the stored artifacts, but not why. Every U record therefore receives one reason. A missing state query means the needed backend or environment state was never queried or saved. An unobservable side effect means the action was taken but the consequence, such as delivery, receipt, payment, or send status, is absent from the artifacts. An ambiguous identity mapping means the trace does not uniquely identify the record or recipient acted on. Missing state-preservation evidence means the native claim requires unrelated state to remain unchanged, but no diff or snapshot is available. Paired-arm asymmetry means a paired case needs both arms to be decidable, but only one is. Evaluator-output ambiguity means the released evaluator emits a label, but the stored evaluator inputs do not determine that label under the benchmark claim. Claim-scope mismatch means the trace supports a related task, but not the claim being reported. Appendix B.1 gives representative examples and the priority rule used when more than one reason could apply.

4 Experimental Setup

4.1 Why these domains

The study uses four main domains chosen to make the evidence claim falsifiable. A method that only finds large gaps is not informative. We therefore mix evidence-sensitive and strong-evaluator domains.

Appendix studies use ANDROIDWORLD and WORKARENA as additional negative controls and OSWORLD-VERIFIED as a higher-noise stress test; they are not part of the main four-domain measurement report.

4.2 Agents, manifests, and fixed workload

The main study uses four fixed agents, Agent A, Agent B, Agent C, and Agent D, selected before scoring as non-redundant measurement probes spanning source openness, scale, and tool-use style. Four probes cannot exhaust a factorial design; they are not meant to be a leaderboard. They are used to test whether evidence uncertainty is structured by domain rather than by a single agent family. Concrete model identifiers and version pins are released with the manifests.

Table 1: Main domains. Each domain contributes 100 case units crossed with four agents, for 400 records. AGENTDOJO stores 800 episodes because each case unit has benign and injected arms; the other domains store 400 episodes.

Domain	Role in the study	Native outcome signal	Stored evidence used
AGENTDOJO	Evidence-sensitive domain with paired utility and security claims.	utility/security label	workspace state across both arms; tool calls; files; messages
APPWORLD	Strong-evaluator control.	database-state unit tests	database state; API logs; unit-test artifacts; native field checks
WEBARENA-VERIFIED	Strong-evaluator control.	verified deterministic evaluator	browser artifacts; network trace; structured final output; verifier inputs
τ^3 -bench retail	Repaired retail benchmark with residual identity and policy uncertainty.	native success signal	database state; tool records; policy-relevant backend evidence

187 The manifest selects 100 eligible case units per domain using a deterministic, predeclared hash
 188 order after excluding smoke-test cases. With four agents this gives 1600 record slots. Because
 189 each AGENTDOJO record runs a benign and an injected arm, the main study stores 2000 execution
 190 episodes. If a domain has fewer than 100 eligible verified cases under the official split, the manifest
 191 uses all eligible cases and records the exception before main scoring.

192 **Denominator rule.** Only infrastructure or pre-run failures are excluded from the evaluation set.
 193 Examples include a sandbox that fails to boot before the task is presented, a missing benchmark
 194 asset, or an external service outage that prevents normal benchmark execution. Agent-caused invalid
 195 actions, tool misuse, timeouts after task start, malformed final answers, or benchmark-facing aborts
 196 are not excluded. They are counted as FAIL when the native benchmark would fail the run, and they
 197 are also reported in the all-attempt audit in Table 10. This separation prevents evidence filtering from
 198 raising scores by removing difficult or agent-induced failures.

199 **Sample size and precision.** The primary unit for uncertainty is the case unit, not the record. Each
 200 domain has 100 case units crossed with four agents, and the four records attached to a case unit can
 201 be correlated because the agents face the same task. We therefore avoid formal precision claims based
 202 on 400 independent records per domain. A record-level binomial calculation would be, at best, an
 203 independence lower bound and is not used for inference. All resampling intervals are case-clustered
 204 and condition on the stored traces.

205 4.3 Drafting and fixing evidence requirements

206 For every case unit, an LLM drafter receives the task text, official policy when present, evaluator code
 207 or description, relevant database/API/browser/ file/tool schemas, and the available trace and post-run
 208 artifact schema. It proposes a candidate *case-level evidence requirement*: the benchmark claim,
 209 the source of that claim, required artifacts, success and fail rules, an UNRESOLVE rule, and whether
 210 any requirement is a stronger measurement claim. The requirement does not supply a ground-truth
 211 answer; it specifies what stored evidence would be sufficient to decide the benchmark’s own claim.

212 Two researchers then cross-validate the candidate requirements in two review rounds before any agent
 213 trace is scored. In each round, they source-check every requirement against the fixed hierarchy in
 214 Section 2: evaluator semantics, then task text and policy, then schema constraints needed to interpret
 215 evaluator-visible state. Requirements that fail the check are manually edited or regenerated and
 216 reviewed again. After the final requirements are locked, neither the LLM nor the human reviewers
 217 may alter them based on agent outcomes. Full prompt fields and review metadata are reported in
 218 Appendix D.

Table 2: Pre-registered predictions used to test calibration. Each prediction is supported, falsified, or inconclusive under the case-cluster interval rule.

Prediction	Expected pattern	Falsification criterion
P1	APPWORLD and WEBARENA-VERIFIED bound widths are each below 15%.	Either lower 95% case-resampling bound exceeds 15%.
P2	AGENTDOJO bound width exceeds the larger negative-control width by at least 20 percentage points.	The upper 95% case-resampling bound of the contrast is below 20 percentage points.
P3	τ^3 -bench retail bound width lies above both negative controls and below AGENTDOJO.	Either contrast interval is entirely in the opposite direction.
P4	At least 75% of non-identified pairwise comparisons occur on AGENTDOJO or τ^3 -bench retail.	The upper 95% case-resampling bound for that fraction is below 75%.

219 4.4 Pre-registered predictions and the falsification clause

220 To make the main claim falsifiable, we register predictions before scoring any record. Predictions are
 221 stored verbatim in the manifest and are not edited after the first scoring run. The prediction target is
 222 bound width, which equals the UNRESOLVE share.

223 Each prediction is evaluated in a three-state way under the case-population interpretation described
 224 in Section 3. A prediction is *supported* only when the relevant 95% case-cluster bootstrap interval
 225 is entirely on the predicted side of the threshold or contrast. It is *falsified* only when the interval
 226 is entirely on the opposite side. It is *inconclusive* otherwise. Point estimates are still reported for
 227 transparency, but they do not alone determine support or falsification.

228 P1 is the calibration check. The method earns its claim only if it produces narrow bounds on strong-
 229 evaluator negative controls. P2–P4 test whether wide bounds and non-identified rankings concentrate
 230 in the domains where the fixed claims are expected to outrun available evidence.

231 4.5 Domain evidence sources

232 Each case unit has fixed evidence requirements. In APPWORLD, the requirements center on database
 233 state, API call history, and unit-test artifacts. In WEBARENA-VERIFIED, they center on verified
 234 evaluator inputs, browser artifacts, and network-trace evidence. In τ^3 -bench retail, they center on
 235 tool logs, backend state, policy-relevant records, and identity resolution. In AGENTDOJO, they
 236 center on workspace state across benign and injected arms, messages, files, tool calls, and the paired
 237 security/utility condition.

238 The scorer never invents a post hoc claim. It asks whether the fixed benchmark claim can be decided
 239 from stored evidence. If not, the record is UNRESOLVE even when the trajectory looks plausible.

240 4.6 Audit and rerun checks

241 We run two fixed checks on the main domains.

242 **Rerun subset.** We rerun Agent A on ten case units per domain, giving 40 rerun case units and
 243 50 rerun episodes because AGENTDOJO reruns paired arms. We report agreement with the original
 244 counting decision and agreement in evidence category (counted success, counted fail, UNRESOLVE
 245 reason), together with exact (Clopper–Pearson) 95% binomial intervals. We also report the absolute
 246 change in lower endpoint, upper endpoint, and width on the rerun subset. This check only probes
 247 stability of evidence labels under a small fixed rerun. It does not support confidence intervals for
 248 agent performance under repeated executions.

249 **Blinded human audit.** We audit 100 main-study records, 25 per domain. Within each domain,
 250 the sample is stratified into counted records, records labeled UNRESOLVE, and records where native
 251 reporting and evidence reporting disagree whenever all three buckets exist. This is a stress sample for
 252 inspectability, not a simple random sample. The raw agreement rates and Cohen’s κ values therefore
 253 do not estimate population-level agreement unless design weights are applied. We report the stratum
 254 counts for every domain.

Table 3: Main measurement report. Counts are over completed records after excluding only infrastructure/pre-run failures. Total $N = S + F + U$. Coverage is $(S + F)/N$. Counted-only score is $S/(S + F)$, a conditional diagnostic over decidable records. Lower is S/N , Upper is $(S + U)/N$, and Width is U/N . The headline performance object is the interval [Lower, Upper], not Counted-only score alone.

Domain	Total	SUCCESS	FAIL	UNRESOLVE	Coverage	Counted-only score	Lower	Upper	Width
AGENTDOJO	400	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
APPWORLD	400	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	400	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
τ^3 -bench retail	400	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
All domains	1600	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL

For each audit item, the auditor sees the task, the trace, available evidence, and the fixed evidence requirements. The auditor does not see the agent identity, native benchmark label, counting decision, UNRESOLVE-reason tag, or alternate view verdicts. Two auditors independently classify every item. We compute:

- auditor-vs.-scorer agreement on the three outcome labels (*supported success*, *supported failure*, and *unresolved*);
- inter-rater Cohen’s κ on {counted, UNRESOLVE};
- inter-rater Cohen’s κ on {R1, ..., R7}, computed only over items both auditors mark UNRESOLVE.

The κ values are descriptive. They can be unstable with 25 audit items per domain, and the taxonomy κ may be computed on a smaller subset because it requires both auditors to choose UNRESOLVE. The release therefore includes item-level audit labels and stratum counts. If a population-level agreement estimate is needed, the same labels can be combined with design weights from the main manifest.

Adoption note. Adopting the method on top of an existing benchmark requires writing one fixed claim and one list of evidence requirements per case unit, then running the rescorer over stored traces. The rescorer returns S, F, U , coverage, counted-only score, lower and upper bounds, bound width, and UNRESOLVE-reason records. Per-domain human-time breakdowns appear in Appendix C.

5 Main Results

Across the fixed four-domain study, the evidence rule separates what a benchmark executed from what its stored artifacts can support as a binary claim. The core result is not a new leaderboard. It is a calibrated reporting pattern: evidence-supported bounds widen where benchmark claims outrun stored evidence and remain narrow where released evaluators already decide the claim.

5.1 Result 1: Evidence uncertainty is concentrated in evidence-sensitive domains

Table 3 is the main measurement report. It reports the fixed set of completed records, the three evidence states, coverage, counted-only score, and the lower and upper evidence-supported bounds. The reviewer-facing test is whether bound width appears broadly, which would suggest an over-conservative reporting layer, or concentrates in the evidence-sensitive domains where unresolved evidence was predicted.

The registered predictions in Table 4 make this calibration claim falsifiable. P1 is the key guardrail: if APPWORLD or WEBARENA-VERIFIED show large bound width, the reporting layer has failed to recognize strong native verification rather than discovered a new benchmark weakness.

5.2 Result 2: Counted-only scores are conditional on decidability

The counted-only score $S/(S + F)$ remains useful because it says how agents perform on records for which the benchmark can decide the claim. It is not the headline all-record estimate when $U > 0$.

Table 4: Pre-registered prediction outcomes. The prediction target is bound width, equivalently the UNRESOLVE rate. Outcomes use the three-state case-resampling interval rule in Section 4.4: supported, falsified, or inconclusive.

Prediction	Registered criterion	Observed quantity	Outcome
P1	APPWORLD and WEBARENA- VERIFIED bound widths each below 15% under the case-resampling interval rule	FILL	FILL
P2	AGENTDOJO bound width exceeds max negative-control bound width by at least 20 pp under the case- resampling interval rule	FILL	FILL
P3	τ^3 -bench retail bound width lies above both negative controls and be- low AGENTDOJO under the case- resampling interval rule	FILL	FILL
P4	At least 75% of non-identified pairs occur on AGENTDOJO or τ^3 -bench retail under the case-resampling in- terval rule	FILL	FILL

Table 5: Directional pairwise robust-ranking matrix. Four agents induce six pairwise comparisons per domain. A cell $A > B$ is reported only when $LB_{A,d} > UB_{B,d}$; ? denotes bound overlap. An asterisk marks unadjusted case-resampling stability, not statistical significance. It is added only when the observed bounds are separated and the lower 2.5th percentile of the case-cluster bootstrap dominance margin remains positive.

Domain	A-B	A-C	A-D	B-C	B-D	C-D	Non-identified pairs
AGENTDOJO	?	?	?	?	?	?	FILL
APPWORLD	?	?	?	?	?	?	FILL
WEBARENA-VERIFIED	?	?	?	?	?	?	FILL
τ^3 -bench retail	?	?	?	?	?	?	FILL
All domains	Unsupported native strict orderings: FILL; native conflicts: FILL						FILL

Table 3 therefore always reports it next to coverage, lower bound, upper bound, and width. A high counted-only score with low coverage means “high success on the decidable subset, low measurement support,” not a strong all-record agent result. Descriptive case-resampling intervals for the same quantities appear in Appendix B.

5.3 Result 3: Some native rankings are not evidence-identified

Evidence-supported bounds also change how agent rankings are reported. Table 5 reports a directional ordering only when one agent’s lower bound exceeds another agent’s upper bound. A question mark means the ordering is not identified from stored evidence, not that the agents are tied. This distinction is central for benchmark reporting: a native point score can impose a strict order even when the evidence-supported bounds do not support any strict order.

5.4 Result 4: Counting decisions are inspectable

The evidence rule is useful only if its decisions are inspectable. The rerun subset and blinded human audit in Table 6 test that property directly. Disagreements are not hidden; they identify cases where the fixed evidence requirements or UNRESOLVE taxonomy needs clarification.

Table 6: Stratified audit and rerun checks. Audit strata report how many items come from counted records, records labeled UNRESOLVE, and native/evidence disagreement records. Agreement and κ values are descriptive for this stratified stress sample, not unweighted estimates for the full record population. Taxonomy κ uses {R1–R7} and is computed only on items both auditors marked UNRESOLVE. Rerun agreement is a small stability check for evidence labels, not an agent-performance uncertainty estimate.

Domain	Audit items	Audit strata	Audit agreement	Countability κ	Taxonomy κ	Rerun agreement / dominant disagreement
AGENTDOJO	25	FILL	FILL	FILL	FILL	FILL; FILL
APPWORLD	25	FILL	FILL	FILL	FILL	FILL; FILL
WEBARENA-VERIFIED	25	FILL	FILL	FILL	FILL	FILL; FILL
τ^3 -bench retail	25	FILL	FILL	FILL	FILL	FILL; FILL
All domains	100	FILL	FILL	FILL	FILL	FILL; FILL

6 Discussion

This paper offers an evidence-aware reporting method, not a full theory of benchmark validity and not a replacement for native evaluators. Its claim is narrow: when stored artifacts do not decide some completed records under the benchmark’s own claim, the benchmark should report the resulting measurement uncertainty rather than collapse it into a single all-record point score.

The method also does not redefine benchmark tasks. The guardrail is simple: required evidence must be grounded in evaluator semantics, task text, policy, or schema. If a requirement goes beyond the benchmark’s own claim, it is labeled as a stronger measurement claim and reported separately. This distinction prevents the reporting layer from silently turning a benchmark into a stricter benchmark.

The bounds should not be confused with confidence intervals. They are induced by unresolved evidence; case-cluster bootstrap intervals answer a different question about finite-case resampling conditional on stored traces. This is why APPWORLD and WEBARENA-VERIFIED are important calibration checks: narrow bounds on strong evaluator domains show that the layer recognizes when released artifacts already decide the claim, while large widths there would falsify the evidence requirements.

The main limits follow from the same design. A vague benchmark claim cannot be rescued by formal bounds; the seven-category UNRESOLVE taxonomy is a stable vocabulary for this study rather than a universal ontology; four agents are measurement probes rather than a definitive leaderboard; and repeated execution uncertainty is outside the main intervals. The practical implication is still direct: benchmark reports should include supported successes, supported failures, unresolved records, coverage, counted-only score, lower and upper bounds, bound width, and inspectable reason labels.

7 Conclusion

Interactive-agent benchmarks often report scores over executed tasks, but execution is not the same as evidence-supported measurement. We introduced a reporting layer tied to each benchmark’s own claim: it classifies completed records as supported success, supported failure, or UNRESOLVE, then reports the evidence-supported bounds implied by unresolved records. Across AGENTDOJO, APPWORLD, WEBARENA-VERIFIED, and τ^3 -bench retail, the registered pattern tests whether uncertainty concentrates in evidence-sensitive domains and remains narrow on strong-evaluator controls. Future benchmark reports should include supported successes, supported failures, unresolved records, and bounds, so that a score is read together with the evidence that can actually support it.

References

- [1] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research, 2024. URL <https://arxiv.org/abs/2412.05467>.
- [2] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024. URL <https://arxiv.org/abs/2406.13352>.
- [3] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks?, 2024. URL <https://arxiv.org/abs/2403.07718>.
- [4] Batu Guan, Xiao Wu, Yuanyuan Yuan, and Shaohua Li. Is your benchmark (still) useful? dynamic benchmarking for code language models, 2025. URL <https://arxiv.org/abs/2503.06643>.
- [5] Amine El Hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. Webarena verified: Reliable evaluation for web agents. SEA @ NeurIPS 2025 poster, 2025. URL <https://openreview.net/forum?id=94t1GxmQkN>. OpenReview non-archival submission.
- [6] Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, Zhiyi Ma, Tristan Thrush, Sebastian Riedel, Zeerak Waseem, Pontus Stenetorp, Robin Jia, Mohit Bansal, Christopher Potts, and Adina Williams. Dynabench: Rethinking benchmarking in NLP. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4110–4124. Association for Computational Linguistics, 2021. URL <https://aclanthology.org/2021.naacl-main.324/>.
- [7] Charles F. Manski. *Partial Identification of Probability Distributions*. Springer, 2003.
- [8] OpenAI. Introducing SWE-bench Verified. Blog post, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. Accessed: 2026-05-01.
- [9] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.
- [10] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2025. URL <https://arxiv.org/abs/2405.14573>.
- [11] Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. Judging the judges: A systematic study of position bias in llm-as-a-judge, 2024. URL <https://arxiv.org/abs/2406.07791>.
- [12] Sierra Research. τ^3 -Bench: Advancing Agent Benchmarking to Knowledge and Voice. Research release, 2026. URL <https://sierra.ai/resources/research/tau-3-bench>. Accessed: 2026-04-30.
- [13] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents, 2024. URL <https://arxiv.org/abs/2407.18901>.

- 381 [14] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao,
382 Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan
383 Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking
384 multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
385
- 386 [15] Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Rethinking
387 benchmark and contamination for language models with rephrased samples, 2023. URL
388 <https://arxiv.org/abs/2311.04850>.
- 389 [16] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang,
390 Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica.
391 Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL <https://arxiv.org/abs/2306.05685>.
392
- 393 [17] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng,
394 Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic
395 web environment for building autonomous agents, 2023. URL <https://arxiv.org/abs/2307.13854>.
396
- 397 [18] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash
398 Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala
399 Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon,
400 Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy
401 Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks,
402 2025. URL <https://arxiv.org/abs/2507.02825>.

A Per-Agent Bound Breakdown

The main tables report domain-level aggregates. This appendix reports the per-agent bounds in each domain, together with case-cluster bootstrap 95% case-resampling intervals for lower endpoint, upper endpoint, width, and coverage under the eligible-case-population interpretation. The per-agent bounds underlie the robust pairwise rankings summarized in Table 5.

Table 7: Per-agent evidence-supported bounds by domain. Each cell reports [LB, UB], width, and counted-only score.

Domain	Agent A	Agent B	Agent C	Agent D
AGENTDOJO	FILL	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	FILL	FILL	FILL	FILL
τ^3 -bench retail	FILL	FILL	FILL	FILL

B Supplemental Result Tables and Examples

This appendix contains reporting views and longer result tables that support the main claims without carrying the main narrative.

B.1 UNRESOLVE Reason Labels

A record labeled UNRESOLVE is tagged with exactly one upstream blocking reason. The categories are not meant as a universal ontology for all future benchmarks; they are the stable reason set used by the scorers and auditors in this study.

Evidence-sensitive record (τ^3 -bench retail)	Negative-control record (AppWorld)
Task Update the intended customer’s mailing address to “123 Maple St.” Benchmark claim The intended customer record has its address updated. Available evidence Tool call <code>update_address(...)</code>; assistant message “address updated.” Missing piece No post-update backend query; the trace also lacks a unique customer id, so target identity cannot be verified. Counting decision <i>unresolved</i> (R3 upstream over R1). Scope guardrail A no-collateral-change check would be reported only if required by the native task or as a separate stronger measurement claim. Effect on report The record stays in N, contributes one unit to U, and widens $[S/N, (S + U)/N]$.	Task Add a \$42.50 grocery expense from yesterday to the expenses tracker. Benchmark claim New row in <code>expenses</code> with amount 42.50, category “groceries,” and correct date, under the released unit-test semantics. Available evidence Post-run database snapshot and unit-test fixture that directly checks the relevant fields. Missing piece None for this trace under the benchmark claim. Counting decision <i>supported success</i>. Effect on report The record contributes one unit to S, appears in the counted-only score, and does not widen the bounds.

Figure 1: Two records walked through the method. A record assigned *unresolved* is not a fail, not a success, and not removed; it widens the evidence-supported bounds. The example also shows the benchmark-claim guardrail: stronger checks are not silently folded into the benchmark-claim result.

C Cost of Adopting the Method

The method is only useful if benchmark builders can actually adopt it. We track the human time required to instantiate the evidence-reporting layer on each main domain.

Predeclaration is the most expensive step but is one-time per case unit. Scoring additional records on the same case unit is cheaper because the fixed requirements are reused. On strong-evaluator domains, much of the work is a documentation refactor: making the verifier’s implicit assumptions explicit.

Table 8: UNRESOLVE-reason labels. The upstream-priority rule assigns the reason that, if resolved first, would unblock the other missing evidence.

UNRESOLVE reason	What is missing	Representative example
R1: Missing state query	The benchmark’s own claim depends on a backend or environment state value that was never queried or stored.	APPWORLD task expects a row added to a CRM table; trace contains no read of that table after the action.
R2: Unobservable side effect	The action is performed, but its consequence, such as delivery, send, payment, or receipt, is not exposed in any artifact.	AGENTDOJO agent appears to send a confirmation, but the messaging tool returns no receipt or message-id.
R3: Ambiguous identity mapping	Multiple records or recipients match the claim and the trace does not pin down which one was acted on.	τ^3 -bench retail task asks to update a customer’s address; two customers with matching name exist, and no unique id is logged.
R4: Required state-preservation evidence absent	The benchmark’s own task, policy, evaluator, or reported claim requires non-modification of unrelated state, but no diff or snapshot evidence is available.	A task requires editing one calendar event and preserving all other events, but no post-run diff over other events is captured.
R5: Paired-arm asymmetry	A paired case requires both arms to be decidable; one arm is decidable and the other is not.	AGENTDOJO benign arm has clean state-diff evidence; injected arm’s security claim is undecidable.
R6: Evaluator output ambiguity	The released evaluator emits a label, but its inputs do not uniquely determine that label under the fixed benchmark claim.	WEBARENA-VERIFIED verifier accepts a string match, but the stored verifier inputs do not show whether the required structured target was reached.
R7: Claim-scope mismatch	The trace shows the agent solving a related task that does not match the specified claim, with no evidence on the claim actually being reported.	τ^3 -bench retail agent gives a semantically helpful refund response, but the policy-relevant backend record required by the benchmark claim is absent.

Table 9: Descriptive case-resampling intervals for evidence-supported bound quantities. The bounds themselves are induced by records that are undecidable from stored evidence. The 95% intervals are case-cluster bootstrap intervals for the eligible-case population interpretation, conditional on stored traces and fixed agent versions. They do not include repeated-execution randomness, API drift, or environment volatility.

Domain	Lower S/N (95% case-resampling interval)	Upper $(S+U)/N$ (95% case-resampling interval)	Width U/N (95% case-resampling interval)	Counted-only score $S/(S+F)$ (95% case-resampling interval)	Native score in-side bounds
AGENTDOJO	FILL	FILL	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	FILL	FILL	FILL	FILL	FILL
τ^3 -bench retail	FILL	FILL	FILL	FILL	FILL
All domains	FILL	FILL	FILL	FILL	FILL

D Evidence-Requirement Drafting Details

The requirements drafter is recorded as part of the experimental manifest rather than inferred from the manuscript text. In the main study, the drafter model is FILL, queried with temperature FILL using prompt version FILL. The drafter is used only to propose evidence requirements before scoring. It does not see agent identities, agent traces, native scores, or outcome labels.

Table 10: Denominator audit. Only infrastructure or pre-run failures are excluded before forming the evaluation set. Agent-caused aborts, invalid actions, and timeouts after normal benchmark execution starts are counted as FAIL when the native benchmark would fail them, and they remain visible in the all-attempt audit.

Domain	Attempted	Infra/pre-run excluded	Completed	Agent-caused FAIL	Notes
AGENTDOJO	FILL	FILL	400	FILL	Paired benign/injected arms remain linked at the case-unit level.
APPWORLD	FILL	FILL	400	FILL	Native database-state failures remain in the evaluation set.
WEBARENA-V	FILL	FILL	400	FILL	Browser-task timeouts after start remain in the evaluation set.
τ^3 -bench retail	FILL	FILL	400	FILL	Tool misuse and policy-path failures remain in the evaluation set.
All domains	FILL	FILL	1600	FILL	Exclusions are not allowed to depend on whether the agent likely succeeded.

Table 11: Pairwise dominance margins for bound-separated comparisons. For a reported ordering $i > j$, the margin is $M_{i>j} = \text{LB}_{i,d} - \text{UB}_{j,d}$. An asterisk marks unadjusted case-resampling stability, not statistical significance. Because the tag is post-selection and unadjusted over 24 comparisons, it should be read as descriptive stability rather than a formal significance test.

Domain	Pair	Bound decision	Margin and 95% case-resampling interval	Interpretation
FILL FROM SCORED MANIFEST	—	—	—	Include every bound-separated pair and its case-cluster bootstrap dominance margin.

The prompt template requires the drafter to separate evidence tied to the benchmark claim from stronger measurement evidence. The released prompt version contains the following fields:

```

You are drafting evidence requirements tied to an interactive-agent benchmark's
own claim.

Inputs:
- task text
- official policy, if any
- evaluator code or evaluator description
- database/API/browser/file/tool schema
- trace schema and available post-run artifacts

Rules:
1. State the benchmark's native success claim.
2. Cite the source of each required evidence item:
   evaluator, task text, policy, or schema.
3. Use the fixed source hierarchy:
   evaluator semantics > task text/policy > schema constraints.
4. Do not add requirements from intuition alone.
5. If a requirement goes beyond the benchmark claim, label it as
   stronger_measurement and keep it separate from the native_aligned claim.
6. Do not infer outcomes. Do not use agent identity, traces, or scores.

Return:
claim_text: ...
native_sources: ...
required_artifacts: ...
success_rule: ...
fail_rule: ...
unresolve_rule: ...
claim_scope: native_aligned | stronger_measurement
minimality_rationale: ...

```

Table 12: Dominant UNRESOLVE reasons by domain. The full UNRESOLVE-reason histogram is released with the case-level manifest; this table reports the largest source of undecidable evidence in each main domain.

Domain	Top UNRESOLVE reason
AGENTDOJO	FILL
APPWORLD	FILL
WEBARENA-VERIFIED	FILL
τ^3 -bench retail	FILL

Table 13: Reporting views used in the paper. The evidence-supported bounds are the headline object. Point scores are diagnostics and are never treated as unbiased all-record performance estimates when records labeled UNRESOLVE exist.

View	Denominator	Undecidable-record handling	Purpose
Evidence-supported bounds	Completed records	Bounds $[S/N, (S + U)/N]$	Headline measurement uncertainty induced by records that cannot be decided from stored evidence
Counted-only score	Counted records only	Excluded from the point score, but reported as UNRESOLVE	Diagnostic success rate conditional on decidability; not all-record performance
Native benchmark	Benchmark default	Whatever the released benchmark reports	Compare against what a user would normally publish
Optimistic endpoint	Completed records	UNRESOLVE $\rightarrow$ success-supporting	Upper evidence-supported bound
Pessimistic endpoint	Completed records	UNRESOLVE $\rightarrow$ not success-supporting	Lower evidence-supported bound
Judge-only	Judge-readable records	Judge may assign SUCCESS/FAIL/INCONCLUSIVE	Diagnostic comparison to semantic face judgment

```

459 source_support:
460     evaluator: ...
461     task_or_policy: ...
462     schema: ...

```

E Additional Domains and Stress Tests

The main paper uses four domains because the headline claim is about evidence uncertainty, not benchmark breadth. This appendix adds two further negative controls and one higher-noise stress test.

E.1 ANDROIDWORLD as an additional negative control

ANDROIDWORLD is reported separately because its task-specific setup and success checking make it a useful additional negative control rather than a likely main source of evidence uncertainty. We score a fixed appendix manifest and report the same bound decomposition. Large bound widths would invite the same falsification reasoning as in P1.

E.2 WORKARENA as an additional negative control

WORKARENA is reported separately for a similar reason. These results do not alter the headline claim, but they test whether the method remains stable outside the four main domains.

Table 14: Approximate human-time cost of instantiating the evidence-reporting method on top of an existing benchmark. Times are wall-clock minutes from a trained annotator and exclude execution compute. The LLM-assisted first draft reduces initial writing time but does not replace human review.

Domain	Draft/fix requirements (min/case)	Score evidence (min/record)	Tag UNRESOLVE (min/record)	Per-domain one-time setup
AGENTDOJO	FILL	FILL	FILL	evidence schema for paired arms
APPWORLD	FILL	FILL	FILL	reuse native unit-test scaffolding
WEBARENA-VERIFIED	FILL	FILL	FILL	reuse verifier; document benchmark-claim requirements
τ^3 -bench retail	FILL	FILL	FILL	policy-path manifest from official policy doc

F OSWORLD-VERIFIED as a Higher-Noise Stress Test

We intentionally do not use OSWORLD-VERIFIED as a co-equal main domain. Its practical setup combines richer operating-system tasks with higher engineering noise, including environment volatility and evaluator complexity. Accordingly, the OSWORLD-VERIFIED appendix reports three counts for the fixed appendix manifest:

1. episodes that fail before the benchmark can execute normally because of infrastructure or pre-run problems;
2. episodes that execute but whose benchmark-side evaluator fails or is unstable; and
3. episodes that execute and evaluate but still remain UNRESOLVE under the fixed evidence requirements.

This keeps execution failure, evaluator failure, and evidence uncertainty separate.

Table 15: Evidence-requirement drafting and review metadata. These fields are filled from the fixed manifest and released with the review package.

Field	Value
Drafter model	FILL
Temperature	FILL
Prompt version	FILL
Visible inputs	Task text; official policy; evaluator code or description; relevant schema; trace schema; available post-run artifact types
Hidden inputs	Agent identity; agent trace; native score; outcome label; alternate view verdicts
Human review rule	Two researchers cross-validate every proposed requirement in two review rounds before scoring. Unsupported requirements are edited or regenerated, then rechecked; requirements that go beyond the benchmark claim are labeled as stronger measurement claims.
Source hierarchy	Evaluator semantics first; task text and policy second; schema constraints needed to interpret evaluator-visible state third.

G Judge-Only Scoring Diagnostic

Judge-only scoring is reported as a diagnostic rather than a headline baseline. For each judge-readable completed record, a fixed blinded judge model receives the task, fixed claim, trace, and available artifacts, but not the agent identity, evidence label, or UNRESOLVE reason. The point is not to claim that judge labels are ground truth. It is to identify records where semantic trace judgment produces a verdict even though the benchmark’s stated claim remains undecidable from stored artifacts.

We report judge-only disagreement rates by domain, together with the fraction of records on which judge-only scoring assigns a success/fail label to a record that remains UNRESOLVE under the evidence rule.

H Supporting Benchmark-Maintenance Study

The main paper focuses on fixed traces. This appendix asks whether the same counted-versus-UNRESOLVE distinction also changes how benchmark maintenance is reported.

H.1 Fixed update round

We run one small fixed update round on the four main domains. Each domain contributes six raw proposals, giving 24 raw proposals total. We select three proposals per domain for materialization or execution, giving 12 selected candidates. Because selected AGENTDOJO candidates are paired, the round yields additional paired-arm episodes. The round is reported as a funnel,

proposed $\rightarrow$ selected $\rightarrow$ executed $\rightarrow$ checked $\rightarrow$ {counted, UNRESOLVE},

rather than as a raw proposal total. A proposal becomes benchmark progress only when it becomes a newly countable case unit or narrows the evidence-supported bounds of a previously UNRESOLVE case.

Table 16: Supporting micro-update round.

Domain	Proposed	Selected	Executed	Checked	Counted	UNRESOLVE	Top UNRESOLVE reason
AGENTDOJO	6	3	6	FILL	FILL	FILL	FILL
APPWORLD	6	3	3	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	6	3	3	FILL	FILL	FILL	FILL
τ^3 -bench retail	6	3	3	FILL	FILL	FILL	FILL
Total	24	12	15	FILL	FILL	FILL	FILL

507 H.2 Matched-budget controls

508 We compare the evidence-aware update process against matched-budget controls that receive the
509 same proposal and selection budget. The controls are one-shot generation, evidence-blind generation,
510 and static benchmark refresh. The key question is not which control writes the most candidate tasks,
511 but which one produces the most countable updates or the largest reduction in bound width.

512 I Responsible Release Plan

513 Interactive-agent evaluations may touch state-changing workflows, security policies, user records, and
514 communication channels. Public artifacts should expose enough information to make denominators
515 and bounds auditable without releasing exploit-enabling operational details.

- 516 • **Released, fully public.** Per-case manifests with task IDs, fixed claim, evidence requirements,
517 claim-scope label, UNRESOLVE taxonomy code, bound contributions, the rescorer source
518 code and tests, per-domain summary statistics, robust-ranking matrices, denominator audits,
519 and audit instructions.
- 520 • **Released under access control.** Full traces for AGENTDOJO paired arms involving prompt-
521 injection content, and trace fragments that contain placeholder credentials, API tokens, or
522 synthetic personal data, after credential scrubbing and sandbox re-pinning.
- 523 • **Not released.** Live-account credentials, real third-party API keys, and environment artifacts
524 that would let a reader reproduce a side effect against a non-sandboxed service.
- 525 • **UNRESOLVE-record visibility.** Even when full traces are gated, UNRESOLVE records remain
526 visible through case identifier, fixed claim, evidence requirements, taxonomy code, and
527 bound contribution.
- 528 • **Versioning.** The manifest, rescorer, evidence-requirement template, evidence-requirement
529 drafting prompt, and UNRESOLVE taxonomy each carry a semantic version. Reported bounds
530 are always paired with a (*rescorer, manifest, requirements, taxonomy*) version tuple.

531 J Formal Definitions

532 J.1 Trajectory

533 A run produces a trajectory:

```
534 tau = (x_0, a_0, x_1, a_1, ..., x_T)
```

537 where x_t is the observed environment state or observation and a_t is the agent action.

538 J.2 Evidence function

539 An evidence function extracts evidence:

```
540 E(tau, env) -> e
```

543 where e may include logs, receipts, state diffs, hashes, tool-call records, verifier inputs, or other
544 domain-specific measurements.

545 J.3 Evidence rule

546 An evidence rule family maps evidence to verdicts:

```
547 R_k(e, policy) -> {SUCCESS, FAIL, INCONCLUSIVE, NOT_APPLICABLE}
```

550 **J.4 Counting and interval**

551 A completed record is counted only when the executed run, fixed claim, and available evidence jointly
552 satisfy the evidence rule:

```
553 Countable(case, agent, run, evidence, rule) = true  
554
```

556 If countable, the rule returns SUCCESS or FAIL; otherwise the record remains UNRESOLVE with a
557 blocking reason. For any completed record set, the reported all-record performance object is:

```
558 TOTAL = SUCCESS + FAIL + UNRESOLVE  
559 COVERAGE = (SUCCESS + FAIL) / TOTAL  
560 COUNTED_ONLY_SCORE = SUCCESS / (SUCCESS + FAIL)  
561 LOWER = SUCCESS / TOTAL  
562 UPPER = (SUCCESS + UNRESOLVE) / TOTAL  
563 WIDTH = UNRESOLVE / TOTAL  
564  
565
```

566 **K Result Macro Requirements**

567 The final camera-ready build should be produced from the scored manifest. The manifest-
568 generated file outputs/latex/results_macros.tex should set \resultdatatrue and define
569 the domain-level macros used in Tables 3–5. It should also define the requirement-drafting meta-
570 data macros consumed by the manuscript. The separate row files pairwise_margin_rows.tex,
571 per_agent_rows.tex, cost_rows.tex, and update_rows.tex populate the longer appendix
572 tables. This check prevents the manuscript from silently replacing missing empirical results or
573 requirement metadata with synthetic values.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes].

Justification: The abstract and introduction state the reporting method, the fixed four-domain study, and the intended scope. The discussion explicitly limits the claims to evidence-supported reporting rather than benchmark validity or a new leaderboard.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes].

Justification: The Discussion section lists limitations of the locked claim, taxonomy scope, four-agent design, instrument-level UNRESOLVE, and bootstrap interpretation.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A].

Justification: The paper defines a reporting object and counting rule but does not present formal theorem statements or proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Sections on study design, evidence-contract locking, denominator rules, audit checks, and the result-macro contract describe how the reported quantities are produced from the released manifests and rescorer.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes].

Justification: The paper states that manifests, evidence contracts, case cards, taxonomy records, and a rescorer are released; the responsible release plan specifies which artifacts are public and which are access-controlled.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: The paper specifies domains, case-unit selection, agents as fixed probes, denominator handling, contract drafting and locking, resampling, audit, and rerun checks.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes].

Justification: The paper reports case-cluster bootstrap intervals as descriptive resampling diagnostics and explains their assumptions and limitations in Sections 3 and 3.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No].

Justification: The appendix reports human-time adoption costs, but the current manuscript does not yet give full hardware, memory, wall-clock execution, or total compute details for every benchmark run.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes].

Justification: The work uses public or benchmark-provided artifacts and includes a responsible release plan for traces, synthetic personal data, credentials, and access-controlled materials.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes].

Justification: The Discussion and Responsible Release Plan describe benefits for benchmark reporting and risks around state-changing workflows, security-related traces, synthetic personal data, and operational details.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes].

Justification: The Responsible Release Plan separates fully public artifacts, access-controlled traces, and non-released credentials or operational artifacts.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No].

Justification: The manuscript cites the existing benchmarks used in the study, but the current draft does not yet enumerate every asset license and terms-of-use constraint in one place.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.

- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes].

Justification: The paper documents the released manifests, evidence contracts, taxonomy tags, rescorer, denominator audits, and versioning in the E&D-oriented release and responsible release sections.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A].

Justification: The blinded audit is an expert audit of benchmark records and evidence labels, not a crowdsourcing experiment or a study of human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

888 Answer: [N/A].

889 Justification: The work does not collect personal data from human subjects or run a human-

890 subjects experiment; the audit concerns benchmark traces and evidence-label decisions.

891 Guidelines:

- 892 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
- 893 with human subjects.
- 894 • Depending on the country in which research is conducted, IRB approval (or equivalent)
- 895 may be required for any human subjects research. If you obtained IRB approval, you
- 896 should clearly state this in the paper.
- 897 • We recognize that the procedures for this may vary significantly between institutions
- 898 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
- 899 guidelines for their institution.
- 900 • For initial submissions, do not include any information that would break anonymity (if
- 901 applicable), such as the institution conducting the review.

902 **16. Declaration of LLM usage**

903 Question: Does the paper describe the usage of LLMs if it is an important, original, or

904 non-standard component of the core methods in this research? Note that if the LLM is used

905 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

906 scientific rigor, or originality of the research, declaration is not required.

907 Answer: [Yes].

908 Justification: The paper describes the contract-drafting LLM, its inputs, its exclusion from

909 agent traces and outcomes, and the human locking process before scoring.

910 Guidelines:

- 911 • The answer [N/A] means that the core method development in this research does not
- 912 involve LLMs as any important, original, or non-standard components.
- 913 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
- 914 be described.